EX03 Report: Network Topologies, Peer Discovery & Byzantine Agreement

Student Name: Maurits van Staalduinen
Student ID: `c9881018`

Claude Sonnet 5.5 used for handin formating, table formating, and clearer wording

Chosen framework: Zenoh (eclipse-zenoh 1.10.2, python)

Task 1 - Discovering Peers

1.1 - Peer Discovery

peer_node.py, one program per node started with its name and the router endpoint

each node declares a liveliness token with its name and subscribes to the tokens of all nodes with history, so it also gets the tokens that existed before it started

the aws vpc has no multicast, so the nodes connect to a zenoh router (own instance) instead of multicast scouting, no node address is configured

every 2s a node prints the names it knows and how many direct zenoh peer connections it has (router not counted)

A, B, C on 3 ec2 instances, router on a 4th

```
python3 peer_node.py A tcp/172.31.21.32:7447    #same for B and C
```

```
[A 17:56:46] online, zid 71ac3c373d7c30b8ced60d4c51b43c91
[A 17:56:48] join B
[A 17:56:48] peers ['B'], 1 zenoh peer connections
[A 17:56:49] join C
[A 17:56:50] peers ['B', 'C'], 2 zenoh peer connections

[B 17:56:48] online, zid f58c70f457bb771cfa7f948c281b555f
```

```
[B 17:56:48] join A
[B 17:56:50] join C
[B 17:56:50] peers ['A', 'C'], 2 zenoh peer connections

[C 17:56:49] join A
[C 17:56:49] online, zid 17d79eab9cc4f9e30638fe3f42722b2f
[C 17:56:49] join B
[C 17:56:51] peers ['A', 'B'], 2 zenoh peer connections
```

all three find each other, and each has 2 direct connections, so they form a full mesh even though they only know the router

1.2 - Dynamic Behavior

B stopped with kill after 10s and started again 8s later (SIGTERM is handled like ctrl+c, a background process ignores SIGINT)

```
[A 17:57:00] peers ['B', 'C'], 2 zenoh peer connections
[A 17:57:01] leave B
[A 17:57:02] peers ['C'], 1 zenoh peer connections
[A 17:57:10] peers ['C'], 1 zenoh peer connections
[A 17:57:11] join B
[A 17:57:12] peers ['B', 'C'], 2 zenoh peer connections

[C 17:57:01] leave B
[C 17:57:02] peers ['A'], 1 zenoh peer connections
[C 17:57:11] join B
[C 17:57:12] peers ['A', 'B'], 2 zenoh peer connections

[B 17:57:11] online, zid 981b66f5c66eedab5fcdd6040284dbeb
[B 17:57:11] join C
[B 17:57:11] join A
```

> *How does a node learn that another node is available?*

each node declares a liveliness token with its name and subscribes to the tokens of all nodes

when a token is declared the others get a PUT sample (join), history=True also gives the tokens that already existed when the node started

underneath the peers connect to the router, which tells them about the other peers, and they then connect to each other directly

> *What happens when Node B leaves, and what happens when it joins again?*

leave: B undeclares its token, A and C get a DELETE sample (leave B) within 1s and drop to 1 peer connection

the token belongs to B's zenoh session, so if B crashed it would also disappear once the session is detected as lost (link closed or lease timeout), only later

join again: B gets a new session, declares the same token, A and C get a join B, B gets A and C right away from history

the restarted B is a new process with no state, the others only recognise it by its name

> *Your discovery mechanism tells you a node is reachable. Can you trust that a discovered node is who it claims to be, or that its messages are truthful?*

no, the name is just a key expression the node picks itself, any process can declare a token named B

when the old B doesn't stop, and a second B joins with the same name, A and C keep showing peers ['B', 'C'] and can't tell there are two

zenoh can add tls / mutual tls and access control so only nodes with a valid certificate can join, but that only proves who sent a message, not that the content is true, an authenticated node can still lie

---

Task 2 - Exchanging Messages

2.1 - Publish/Subscribe

pubsub_node.py, peer_node.py plus every node publishes a numbered hello message on a key with its own name every 2s and subscribes to the hello keys of all nodes

the sender is the last chunk of the key, a node skips its own messages (the session also gets its own puts)

```
pub = session.declare_publisher(f"mesh/hello/{name}")
sub = session.declare_subscriber("mesh/hello/*", on_hello)
pub.put(f"Hello from Node {name} #{i}")
```

A, B, C on 3 ec2 instances again, only the router endpoint given, ran for 12s

```
python3 pubsub_node.py A tcp/172.31.17.137:7447    #same for B and
C
```

```
[A 18:05:44] recv from B: Hello from Node B #1
[A 18:05:45] sent hello #2, peers ['B', 'C']
[A 18:05:46] recv from C: Hello from Node C #1
[A 18:05:46] recv from B: Hello from Node B #2

[B 18:05:45] recv from A: Hello from Node A #2
[B 18:05:46] recv from C: Hello from Node C #1
[B 18:05:46] sent hello #2, peers ['A', 'C']

[C 18:05:44] online, zid 239822b5b26726e70a032d503fe3ff6e
[C 18:05:44] recv from B: Hello from Node B #1
[C 18:05:45] recv from A: Hello from Node A #2
[C 18:05:46] sent hello #1, peers ['A', 'B']
[C 18:05:46] recv from B: Hello from Node B #2
```

every node gets every hello of the others, in order and without gaps

except A #1: A sent it at 18:05:42, C only started at 18:05:44, so C never got it (B, already running, did)

2.2 - Discussion Questions

> *What is the main difference between static configuration and dynamic discovery?*

static: every node gets the addresses of the others in advance (like ex02, peer as an argument), a new or moved node means changing the config and restarting

dynamic: nodes find each other at runtime and can join and leave anytime (task 1.2), here only the router endpoint is fixed, never another application node

> *Why is the publish/subscribe pattern useful for communication between distributed processes?*

publisher and subscriber don't know each other, they only share a key expression, zenoh delivers to whoever is subscribed right now

so a node can publish without knowing who listens or how many, new nodes just subscribe (C got B's messages right away), no per-peer connections or address lists in the code

it's also asynchronous, the publisher doesn't wait for the receivers and the subscriber gets a callback instead of polling

> *Is peer discovery sufficient to make an application fault-tolerant? Explain briefly.*

no, discovery only tells you who is there right now, it doesn't keep data or make up for failures

messages sent while a node is down or not started yet are lost, a restarted node has lost its state, nothing is retried or replicated

discovery also doesn't detect wrong behavior, a node can be reachable and still send wrong data, fault tolerance needs replication, retries / stored history and agreement on top

the instances were destroyed after the run

Task 3 - Byzantine Generals

3.1 - Three Nodes

byzantine_node.py, role, traitor and number of generals from the command line, same zenoh pub/sub as task 2

commander puts its order on a separate key for each lieutenant, each lieutenant relays what it got on a key with its own name, the commander waits until all lieutenant liveliness tokens exist before sending

traitor commander: attack to one lieutenant, retreat to the other, traitor lieutenant: relays the opposite

decision: majority over the values, a tie goes to the commander's value, a missing relay (10s after the order) counts as retreat

```
from_commander = values.pop("commander", DEFAULT)
votes = [from_commander] + list(values.values())
attack, retreat = votes.count("attack"), votes.count("retreat")
if attack == retreat:
    return from_commander, votes
return ("attack" if attack > retreat else "retreat"), votes
```

every general on its own ec2 instance, lieutenants started first, commander always with --order attack

```
python3 byzantine_node.py L1 --role lieutenant --nodes 3 --connect
tcp/172.31.19.254:7447
python3 byzantine_node.py L2 --role lieutenant --nodes 3 --connect
tcp/172.31.19.254:7447
python3 byzantine_node.py C --role commander --order attack --nodes
3 [--traitor] --connect tcp/172.31.19.254:7447
```

3.2 - Experiments with Three Nodes

all loyal

```
[C 18:16:36] order attack -> L1
[C 18:16:36] order attack -> L2
[L1 18:16:36] votes ['attack', 'attack'] -> decision attack
[L2 18:16:36] votes ['attack', 'attack'] -> decision attack
```

traitor commander

```
[C 18:16:52] order attack -> L1
[C 18:16:52] order retreat -> L2
[L1 18:16:52] recv attack from commander
[L1 18:16:52] recv retreat from L2
[L1 18:16:52] votes ['attack', 'retreat'] -> decision attack
[L2 18:16:52] recv retreat from commander
[L2 18:16:52] recv attack from L1
[L2 18:16:52] votes ['retreat', 'attack'] -> decision retreat
```

all loyal: both attack, agreement and validity hold

traitor commander: L1 attacks, L2 retreats, no agreement

L1 has "commander says attack, L2 says retreat", it gets exactly the same if the commander is loyal (attack) and L2 is the traitor and relays retreat

the two situations give L1 the same input, so it has to decide the same in both

for validity with a loyal commander it has to follow the commander (attack), L2 does the same in its situation (retreat), so they disagree

the other way, a fixed default on conflict (e.g. always retreat), would make them agree here but break validity when L2 lies, with 3 generals one of the two always fails

3.3 - Four Nodes

same program with --nodes 4 and a third lieutenant L3

traitor commander (attack to L1 and L3, retreat to L2)

```
[L1 18:17:32] votes ['attack', 'retreat', 'attack'] -> decision
attack
[L2 18:17:32] votes ['retreat', 'attack', 'attack'] -> decision
attack
[L3 18:17:32] votes ['attack', 'retreat', 'attack'] -> decision
attack
```

traitor lieutenant L2 (gets attack, relays retreat)

```
[L2 18:17:53] recv attack from commander
[L2 18:17:53] relay retreat to the other lieutenants
[L1 18:17:53] votes ['attack', 'retreat', 'attack'] -> decision
attack
[L3 18:17:53] votes ['attack', 'attack', 'retreat'] -> decision
attack
```

all loyal: every lieutenant has ['attack', 'attack', 'attack'] -> attack

decisions of the loyal lieutenants (commander's order attack):

GENERALS	TRAITOR	L1	L2	L3	AGREEMENT	VALIDITY
3	none	attack	attack	-	yes	yes
3	commander	attack	retreat	-	no	-
4	none	attack	attack	attack	yes	yes
4	commander	attack	attack	attack	yes	-
4	lieutenant L2	attack	(traitor)	attack	yes	yes

validity doesn't apply with a traitor commander, there's no "commander's order" to follow

3.4 - Discussion Questions

> *Why does majority voting succeed with three lieutenants but fail with only two?*

with three lieutenants every one holds 3 values and at most one of them comes from the traitor

loyal commander: the commander's value and the other loyal lieutenant's relay are both correct, 2 of 3, so the majority is the commander's order (validity)

traitor commander: all lieutenants are loyal and relay honestly, so every one holds the same 3 values (what the commander sent to L1, L2, L3), same values -> same majority (agreement)

with two lieutenants there are only 2 values, one lie makes it 1:1 and there is no majority to break the tie, in general it needs n > 3f generals, 4 > 3 works, 3 > 3 doesn't

> *What does the commander provide that makes agreement possible with four nodes, and why does the problem become harder when all nodes are equal?*

the commander gives one single value to agree on and a fixed starting point, everyone knows what is being decided and who sends first

the set of generals and the roles are also fixed in advance (--nodes, --role), every lieutenant knows how many values to wait for

with all nodes equal every node has its own value, so the agreement has to be run for every node's value (one "commander" each), and someone has to decide when a round starts and who takes part

the discovery from task 1 doesn't help there, anyone can join under any name, a traitor could join as several nodes and then there are more than n/3 traitors

> *Your four-node implementation guarantees agreement, but can it identify which node is the traitor? Explain what a loyal node can and cannot observe.*

no, a lieutenant only sees what was sent to itself: the commander's order to it and the relays of the others

it cannot see what the commander sent to the other lieutenants

L1 had ['attack', 'retreat', 'attack'] in both 4-node traitor runs, once the commander lied to L2 and once L2 lied in its relay, for L1 it looks exactly the same

L1 only knows "commander or L2 lied", not which one

L2 in the traitor commander run (retreat from the commander, attack from both others) can conclude it was the commander, because with at most 1 traitor L1 and L3 can't both be lying

but it can't prove it to the others, oral messages aren't signed, so its accusation could be a lie as well, with signed messages (Lamport's SM algorithm) a conflicting order could be shown to everyone

> *Where multicast is blocked, how could nodes communicate in practice? Does your middleware offer such a feature?*

instead of scouting, nodes connect to endpoints they already know: a router / rendezvous node, or a few bootstrap peers, and learn the rest from them (like libp2p bootstrap nodes or bitcoin dns seeds)

zenoh has this: connect/endpoints in the config (tcp, udp, quic, tls, websocket), zenohd routers that can be chained across networks, and gossip scouting that spreads the peers over the existing connections

this exercise needed it: aws vpcs don't support multicast, so every node got only the router endpoint, the router told the peers about each other and they connected directly (2 peer connections each in task 1)

no application node address was configured, only the router as infrastructure